DESIGN AND ANALYSIS OF ALGORITHMS

LECTURE 1

Algorithm Fundamentals

minhnth@hus.edu.vn

Lecture outline

- Key concepts
  - Computational Problems
  - Algorithms, Data Structures, and Programs
- Algorithm Analysis and Complexity Overview
- Theory of Computation
  - Turing Machines
  - Primitive Recursive Function
- Exercises

Key concepts

- ✓ Problem
- ✓ Algorithm
- ✓ Data Structure
- ✓ Program

Problem

- Problem in informatics/computer science
  - A class of problems with the same specification.
  - Example:
    - Sort a list $(a,n)=(a_1,a_2,\ldots,a_n)$ not sort the list (4,2,7,3,6,1).
    - Find path from city ***A*** to city ***B*** on a map.
  - **A problem = <Inputs, Outputs>**
- Can the problem be solved?
- How difficult is the problem?
- How to model real-life problems to algorithmic problems?

Algorithm

- Algorithm:

  is a finite sequence of instructions (well-defined, computer-implementable) for solving a problem.

Input $\xrightarrow[\text{Algorithm}]{}$ Output

- Features of algorithms
  - Generality: Apply to any input from a specified set.
  - Finiteness: Terminate after a finite number of steps.
  - Definiteness: Each step is clearly and unambiguously.
  - Effectiveness: Feasible within constraints time and resources.

Algorithm

- Expressing algorithms
  - Natural language
  - Flowcharts
  - Pseudocode
  - Programming Languages
- Example:

Euclid's algorithm

Step 1 If $n = 0$, return the value of m as the answer and stop; otherwise, proceed to Step 2.

Step 2 Divide m by n and assign the value of the remainder to r

Step 3 Assign the value of n to m and the value of r to n. Go to Step 1.

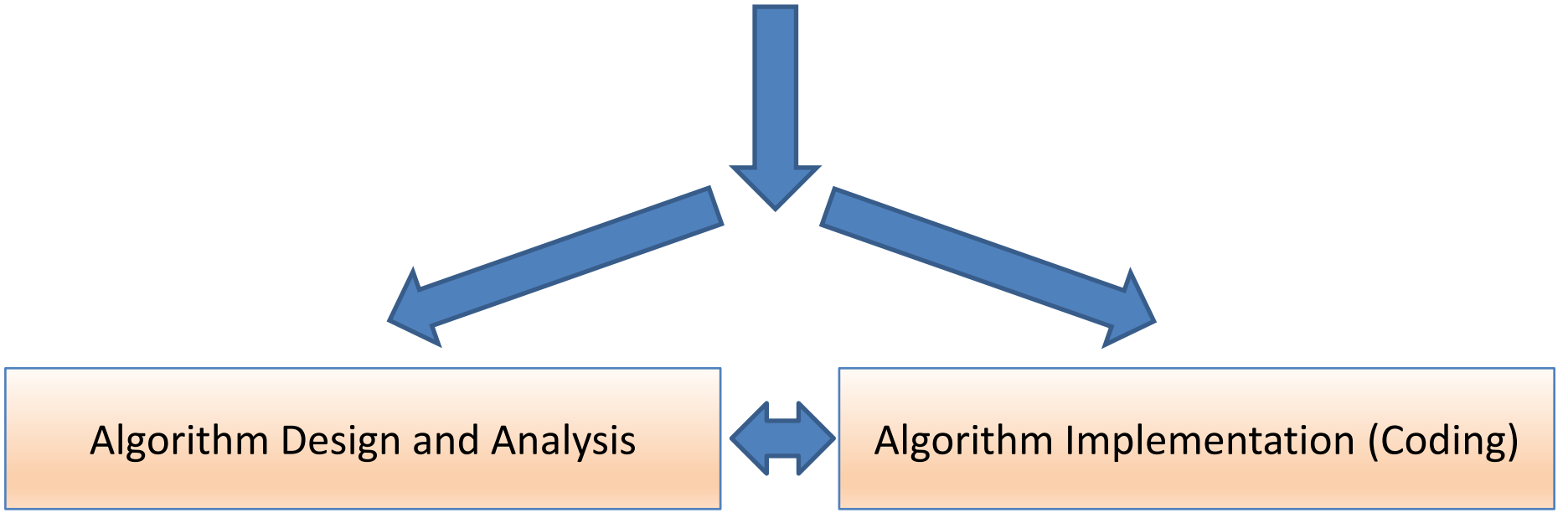
[Anany's book P32]

ALGORITHM *Euclid*(m, n)

//Computes gcd(m, n) by Euclid's algorithm
//Input: Two nonnegative, not-both-zero integers m and n
//Output: Greatest common divisor of m and n
while $n \neq 0$ **do**
  $r \leftarrow m \bmod n$
  $m \leftarrow n$
  $n \leftarrow r$
return m

Algorithm

- Research on algorithms
  - Computability: which problem can be solved by the algorithm.
  - Optimization: Finding more efficient or 'better' algorithms.
  - Implementation: Deploying algorithms into executable code.



Data Structure

- Algorithms operate on data (input, output, intermediate).

  Data organization: Key to algorithm design and analysis.
- A data structure is a systematic way of organizing and storing data to enable efficient processing.
- Abstract data types (ADT):
  - Example: Arrays, Lists, Stacks, Queues, Trees, Graphs,…
  - Key aspects:
    - Specification: Logical definition of data and operations
    - Implementation: Using **Algorithms** to build the structure.
    - Application: Usage in solving specific problems.

Program

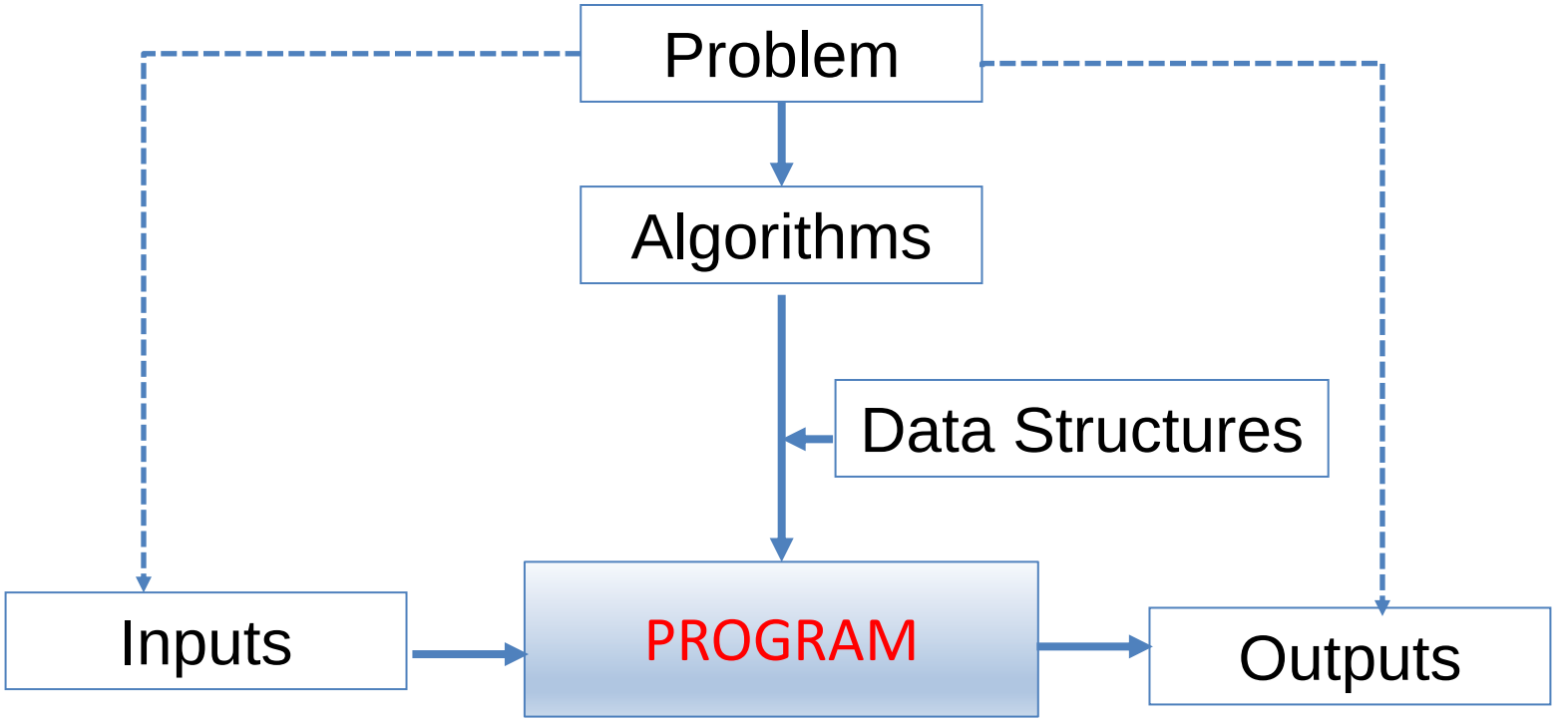
- A computer program is a collection of instructions (written by a programmer) in a programming language to perform specific tasks for solving problem.

Programs = Algorithms + Data Structures

(Niklaus Wirth)

https://en.wikipedia.org/wiki/Niklaus_Wirth

Solving problem on Computer



Algorithm Analysis and Complexity

- ✓ Algorithms Design
- ✓ Algorithms Analysis

Algorithm Design

- Designing algorithms ⇦ How to find suitable algorithm?
  - Algorithms for specific problems.
  - Algorithm design techniques (Strategy/Paradigm/Method)
- Common algorithm design techniques:
  - Divide-and-Conquer – *Chia để trị*
  - Backtracking – *Quay lui*
  - Branch-and-Bound – *Nhánh cận*
  - Greedy – *Tham lam*
  - Dynamic Programming – *Quy hoạch động*
  - Approximation – *Xấp xỉ*
  - *Heuristics*

Algorithm Analysis

- How to verify the algorithm?
  - Proving the Correctness – Chứng minh tính đúng đắn.
- How to compare algorithms?
  - Assessing the Effectiveness – Đánh giá tính hiệu quả.

Two main aspects of efficiency:

- Time efficiency – How fast does the algorithm runs?
  - ⇨ Time complexity – Độ phức tạp thời gian.
- Space efficiency – How much extra memory does it uses?
  - ⇨ Space complexity – Độ phức tạp không gian.

Algorithm Analysis

- Example: Compare two sorting algorithms with $n=10^6$ items.
  - *Insertion sort:* Requires approximately $c_1 n^2$ operations.
  - *Merge sort:* Requires approximately $c_2 n \log n$ operations.

  A Biased Comparison:
  - *Insertion sort* with $c_1=2$, and speed 10^9 ops/sec

    $$T_{Ins} = 2 \,.\, (10^6)^2 \,/\, 10^9 = 2000 \text{ seconds} \approx 33 \text{ minutes}$$
  - *Merge sort* with $c_2=50$, and speed 10^7 ops/sec

    $$T_{Mer} = 50 \,.\, (10^6 \log 10^6) \,/ 10^7 \approx 100 \text{ seconds}$$

  Merge faster than *Insertion* 20 times

  If n increase to 10^7 items the results is 2.3 days vs 13 minutes (!)

Algorithm Analysis

- The complexity of some sorting algorithms

Algorithm	Worst Case	Best Case	In-place	Stable
Selection Sort	$O(n^2)$	$O(n^2)$	Yes	No
Insertion Sort	$O(n^2)$	O(n)	Yes	Yes
Bubble Sort	$O(n^2)$	$O(n^2)$	Yes	Yes
Bubble Sort 2	$O(n^2)$	$O(n)$	Yes	Yes
Quick Sort	$O(n^2)$	$O(n\log n)$	Yes	No
Merge Sort	$O(n\log n)$	$O(n\log n)$	No	Yes
Shuffle Sort (Trộn ngẫu nhiên)	?	?		

Algorithm Analysis

- Two Approaches to Algorithm Analysis
  - Experimental Analysis – *Phương pháp thực nghiệm*

    Process: Code $\rightarrow$ Execute $\rightarrow$ Measure resources (time/memory).

    Pros: Can analyze all programable algorithms.

    Cons: Platform-dependent (hardware & software).
  - Theoretical Analysis – *Phương pháp lý thuyết*

    Process: Model complexity as a function of input data size (n).

    Pros: Platforms-independent.

    Supports large-scale analysis ($n \rightarrow \infty$).

    Cons: Mathematically demanding.

Turing Machine

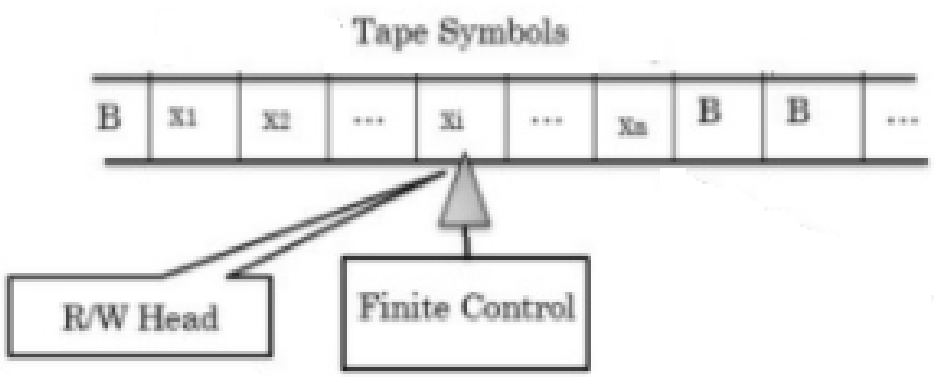
- ✓ Turing Machine Description
- ✓ Turing Machine Example
- ✓ Church-Turing thesis

Turing Machine Description

- A Turing machine:
  - An abstract mathematical model capable of simulating any computer algorithm.
  - Foundation of modern computing (Turing, 1936).
  - https://en.wikipedia.org/wiki/Turing_machine
- Alan Mathison Turing (1912–1954)
  - English mathematician, logician, and cryptanalyst.
  - Father of Computer Science and Artificial Intelligence.
  - https://en.wikipedia.org/wiki/Alan_Turing

Turing Machine Description

- Turing machine structure:
  - Tape: An infinite sequence of cells for reading and writing symbols.
  - Read/Write Head: Moves left or right to read and modify symbols on the tape.
  - Controller: A finite state machine that executes instructions from a transition table.



Turing Machine Description

- Turing machine operation:
  - Set input on the tape.
  - The read/write head moves along the tape in steps by instructions until stop or halt state reached.
  - The output given by symbols on the tape.

Demonstration of Turing Machine operation

(file .flv, open with VLC media player)

Turing Machine Description

- Formal definition:

  Turing machine can be formally defined as a 4-tuple:

  $$M = (K, \Sigma, \delta, s)$$

  Where:

  K - set of states of M. $s, h \in K$ are predefined states (start, halt).

  Σ - set of symbols/characters of M.

  $K \cap \Sigma = \varnothing$ and ▸, ■ $\in \Sigma$ are predefined symbols (begin, end)

  δ is transition function formed as partial function:

  $$\delta : K \times \Sigma \rightarrow \Sigma \times K \times \{\rightarrow, \leftarrow, -\}.$$

  $\delta(t,k) = (k_1, t_1, \{\rightarrow / \leftarrow / -\})$

  - Replace symbol $k \rightarrow k_1$, change state $t \rightarrow t_1$;
  - Move read/write head by direction $\{\rightarrow / \leftarrow / -\}$

Turing Machine Example

- Turing Machine example: $M_1 = (K, \Sigma, \delta, s)$

Where:

$K = \{s,q,h\}$ s – start, h – halt.

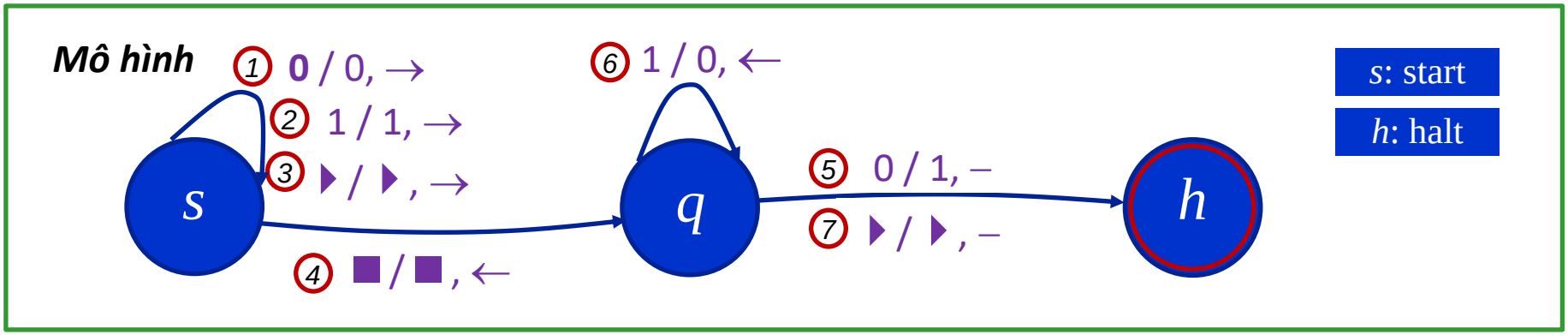
$\Sigma = \{0, 1, ▸, ■\}$ ▸ - begin, ■ - end

Table of transition functions δ :

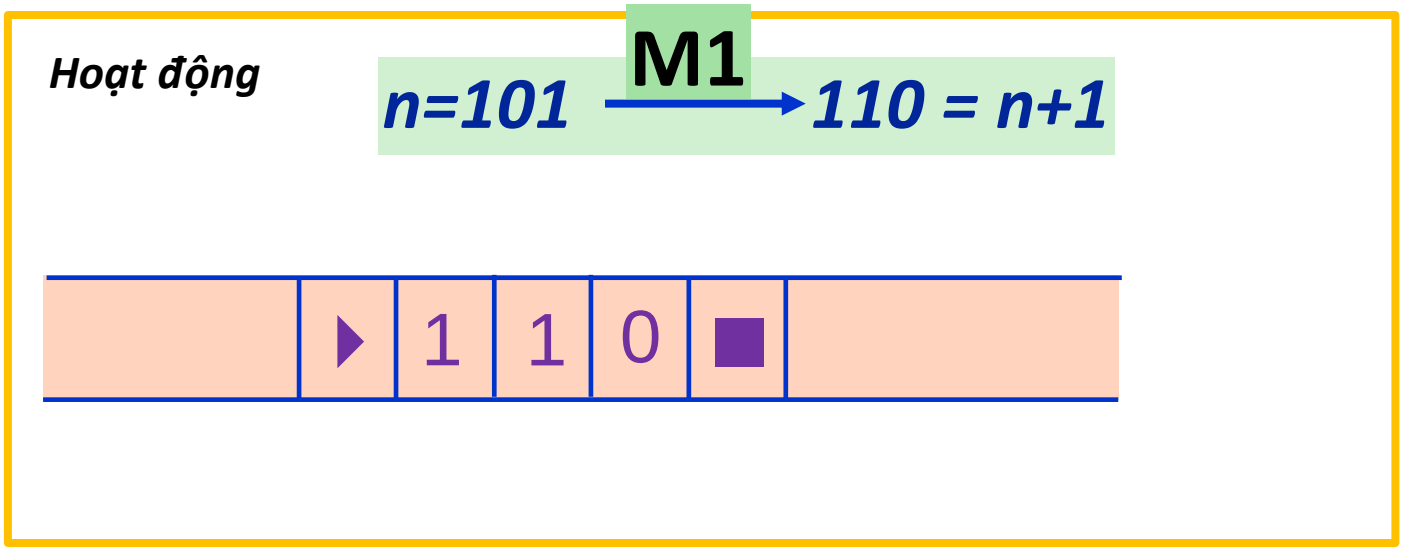
STT	t	k	$\delta(t,k) = (k_1,t_1,\{\rightarrow / \leftarrow / -\})$
1	s	0	(0 , s, $\rightarrow$)
2	s	1	(1 , s, $\rightarrow$)
3	s	▸	(▸ , s, $\rightarrow$)
4	s	■	(■ , q, $\leftarrow$)
5	q	0	(1 , h, –)
6	q	1	(0 , q, $\leftarrow$)
7	q	▸	(▸ , h, –)

Turing Machine Example

- Operation of M_1 **Demo**



Step	Func
1	3
2	2
3	1
4	2
5	4
6	6
7	5
8	-



Turing Machine Example

- Operation of M_1:

  Operate with other inputs:

  - n = 1001 Result: ouput value, number of steps?
  - n = 1011 Result: output value, number of steps?

  Unexpected instance of M_1 and solution: M_1 with input:

  - n = 11 Result: ouput value, number of steps?

    What trouble?

    Improve M_1?

Church -Turing thesis

- The Turing machine as a computational model
  - Can simulate any algorithm performable by a modern computer.
  - Serves as the fundamental model for describing processor functionality.

"Turing Machine ≡ Algorithm"

- Church-Turing thesis and its variations:
  - Formal Statement: A function on natural numbers is computable by an effective method if and only if it is computable by a Turing machine.
  - Algorithmic Variation: Any problem solvable by finite and definite instructions can be solved by a computer program.

https://en.wikipedia.org/wiki/Church–Turing_thesis

Turing Machine and Algorithm

- Turing Machine and Conway's Game of Life

  P. Rendell, *Turing Machine Universality of the Game of Life,* Complexity and Computation 18, Springer 2016, DOI 10.1007/978-3-319-19842-2_2

  Play Game of life online: https://playgameoflife.com/

Primitive Recursive Function

- ✓ Introduction PRF
- ✓ PRF definitions
- ✓ Prove a function is PRF

Primitive Recursive Function

- Definition: Primitive Recursive Function (PRF) are among the number-theoretic functions. These functions take *n* arguments (*n*-ary), which are the natural numbers (nonnegative integers) {0, 1, 2, ...} to the natural numbers.

- Primitive recursive function is computable. Mean exist a Turing machine *M* to compute the value of function *f* if *f* is a PRF.

Reference link:

https://en.wikipedia.org/wiki/Primitive_recursive_function

Primitive Recursive Function

- The basic primitive recursive functions are given:
  - **Constant function**: The 0-ary constant function 0 is PRF.
  - **Successor function**: The 1-ary successor function *S*, which returns the successor of its argument is PRF. S(x) = x + 1.
  - **Projection function**: For every $n \geq 1$ and each i with $1 \leq i \leq n$, the n-ary projection function P_i^n, which returns its i-th argument, is PRF.

    $(P_i^n(x_1,..,x_i,\ldots x_n)=x_i)$

Primitive Recursive Function

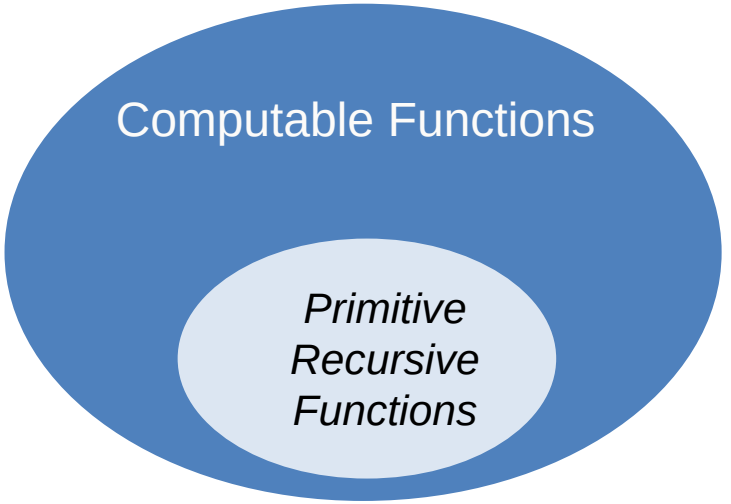
- More complex PRF obtained by applying the operations:
  - **Composition**: Given f is a k-ary PRF, and $g_1,...,g_k$ are k m-ary PRF, the composition of f with $g_1,...,g_k$, i.e. the m-ary $h(x_1,...,x_m) = f(g_1(x_1,...,x_m),...,g_k(x_1,...,x_m))$ is PRF.
  - **Primitive recursion**: Given f, a k-ary PRF, and g, a $(k+2)$-ary PRF, the $(k+1)$-ary function h is defined as the primitive recursion of f and g:

    $h(0,x_1,...,x_k) = f(x_1,...,x_k)$ and

    $h(S(n),x_1,...,x_k) = g(h(n,x_1,...,x_k),n,x_1,...,x_k)$ is PRF

Primitive Recursive Function

- Identifications:
  - PRFs are computable, mean exist a Turing machine or algorithm to compute the value of the function.
  - But set of PRF are not all the computable fuctions



Primitive Recursive Function

- Some provable PRFs:
  - Addition: add(a,b)= $a+b$
  - Multiplication: mul(a,b)= $a \times b$
  - Exponentiation: exp(a,b)= a^b,
  - Factorial: fac(a) = a!
  - Predecessor: pred(a) = (a>0 ? a-1 : 0)
  - Proper subtraction: proper_sub(a,b) = ($a \geq b$? a-b : 0)
  - Minimum: minimum(a_1, ... a_n)
  - Maximum: maximum (a_1, ... a_n)

Primitive Recursive Function

- Rewrite defined definitions about PRF:

  (1) Constant function: C=const

  (2) Successor function: $S(x) = x+1$

  (3) Projection function: $P_i^n(x_1,..,x_i,\ldots x_n)=x_i$

  (4) Composition: If $g_1,...,g_k$ are m-ary PRF and f is k-ary PRF then:

  $h(x_1,...,x_m) = f(g_1(x_1,...,x_m),...,g_k(x_1,...,x_m))$ is PRF.

  (5) Primitive recursion: If f,g are k-ary and k+2-ary, and h is k+1-ary function such:

  $h(0,x_1,...,x_k) = f(x_1,...,x_k)$,

  $h(S(n),x_1,...,x_k) = g(h(n,x_1,...,x_k),n,x_1,...,x_k)$, is a PRF.

Primitive Recursive Function

- Example of proving a function is PRF:

  Addition: add(a,b)= a+b

  Prove add function is a PRF?

  - add(0,b) = 0 + b = b = $P_1^1(b)$ is PRF since P is PRF (1)
  - add(a+1,b) = a+1+b = a+b+1 = add(a,b)+1

    = S(add(a,b)) = S(P_1^3(add(a,b),a,b))

    is PRF since S, P are PRFs and Composition (2)

  - From (1) and (2) get add is PRF by Primitive recursion.

Exercises

- Build Turing Machine
- Prove Primitive Recursive Function
- Problem specification, design algorithm and program.
- More detail in Hw1_AlgorithmFundamental.doc